DEPLOYMENT PACKAGE · CONSOLIDATED REFERENCE

# CFR Agentic AI — Deployment Package

The single reference for deploying the app on **GCP / Databricks / Local**, running it over a client's own data sources (Option B), and switching the LLM provider (**Gemini → OpenAI / Anthropic / …**) — with the current folder structure, architecture, runbook, and verification status.

**Targets:** Local · GCP · Databricks **LLM:** env-switchable (Gemini default) **Mapping:** Option B (dynamic, human-gated)

**Status:** Milestone A complete & verified



## 1 Overview & what it delivers

The app (FastAPI + Google ADK 5-agent orchestration + React/Vite UI) reads a tabular **Silver layer** (BigQuery `tiger_semantic` — 34 views / 569 columns) and writes decisions to `tiger_decisions`. This package makes that same app **portable**:

- **Any platform** — one driver deploys to GCP (Cloud Run) or Databricks, or locally via Docker.
- **Any data source** — a human-gated agent maps a client's schema onto the standard contract and exposes it as **virtual views**, so the app reads it with **zero app-code change**.
- **Any model** — the LLM provider is an **env switch**: Gemini by default, OpenAI / Anthropic / others via ADK's LiteLlm wrapper.

> **Design principle:** every change is additive and backwards-compatible. With no env overrides set, the app behaves exactly as the original GCP build.

## 2 Package structure (current)

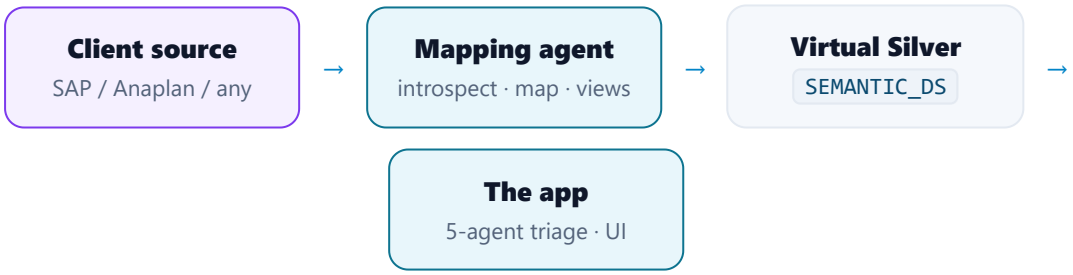
All deploy-time assets live in one dedicated folder; the modules the running app imports stay in `backend/code/` (so they sit on the container's flat `/app` path).

```
deployment/                          # the dedicated package folder
  STEPS.md                           # the runbook (deploy steps + Option B + LLM switch)
  README.md                          # folder index
  data-model.md                      # standard Silver contract (generated)
  config/
    platform.env.example
    gcp/        silver.config.json · env.gcp · deploy.config.json
    databricks/ silver.config.json · env.databricks · databricks.app.yaml · databricks.bundle.yml
  scripts/
    driver.py · driver.sh            # platform-selectable deploy -> live URL
    run_schema_mapping.py            # Option-B mapping runner
    gen_data_model.py                # regenerates data-model.md
  docs/  CFR_DEPLOYMENT_PACKAGE.{html,pdf}   # this document

backend/code/                        # app-imported modules (copied to /app by Dockerfile)
  orchestrator_service/silver_target.py     # data seam (SEMANTIC_DS / DECISIONS_DS)
  orchestrator_service/model_provider.py    # LLM provider factory (env-driven)
  platform_adapters/                        # BigQuery + Databricks catalog adapters
  mapping_tools.py                          # Option-B introspection / mapping tools

deploy.sh · cloudbuild.yaml · deploy-gcp.sh · docker-compose.yml   # repo root (unchanged location)
```

## 3 Architecture — the four seams




| SEAM | WHAT IT DOES | WHERE |
|---|---|---|
| **Data seam** | App reads a config-resolved `{SEMANTIC_DS}.<view>`; a virtual layer swaps in transparently. Env default = the GCP literal. | `silver_target.py` |
| **Platform adapters** | One interface, two impls (BigQuery + Databricks): introspect · profile · render view DDL. | `platform_adapters/` |
| **LLM factory** | Env-driven provider/model selection; Gemini default, others via LiteLlm. | `model_provider.py` |
| **Driver** | Picks platform, exports the resolved target, deploys, smoke-tests, prints the URL. | `scripts/driver.py` |

## 4 Deploy — the 5 steps

Run from the repo root. Prereqs: Python 3.10+ · Docker (local) · `gcloud` (GCP) · `databricks` CLI (Databricks).

1. **Pick platform** — `export DEPLOY_PLATFORM=gcp|databricks|local`
2. **Configure** — edit `deployment/config/<platform>/silver.config.json` (data binding)
3. **Map client data** *(only if the schema differs — else skip)* — see §5
4. **Deploy** — one command, returns the live URL
5. **Verify** — health + data-dictionary checks

```
bash deployment/scripts/driver.sh --platform local      --smoke   # -> http://localhost:3001
bash deployment/scripts/driver.sh --platform gcp        --smoke   # wraps deploy.sh -> Cloud Run URL
bash deployment/scripts/driver.sh --platform databricks           # bundle + apps -> App URL
# --dry-run prints the resolved config + commands without executing.

# Verify:
curl <url>/healthz · /api/health · /api/data-dictionary   # 200 + non-empty "views" = data wired
```

> GCP is also deployed via `deploy.sh` / `cloudbuild.yaml` / `deploy-gcp.sh` — the driver wraps these. They build the backend from `./backend`, so the new modules + (optional) litellm ship automatically.

## 5 Option B — dynamic source mapping

When a client's tables/columns differ from the standard schema, an agent maps them and creates rename-only virtual views named exactly like our 34 standard views — so the app reads them unchanged.

**a · Introspect** → **b · Profile + Propose** → **c · Human gate** → **d · Apply (create views)**

```
py deployment/scripts/run_schema_mapping.py --platform gcp --source <proj.client_raw> --introspect-only
py deployment/scripts/run_schema_mapping.py --platform gcp --source <proj.client_raw>          # propose,
then STOP
# edit schema_mapping_out/mapping_worksheet.md -> confirm rows into proposal.json mapped[]
py deployment/scripts/run_schema_mapping.py --platform gcp --source <proj.client_raw> --apply  # create
views
```

> **Guardrail (in code, not just the prompt):** low-confidence / column-less proposals are demoted to *ambiguous*; uncovered required keys are surfaced as *unmapped*. Nothing is invented and no required column is silently NULL — a human approves before any view is created.

## 6 LLM provider switch (OpenAI & others)

**Default is Gemini (Vertex) — no keys, no change.** Switching to another provider is env-only; one factory (`model_provider.py`) routes all 5 agents + the schema-mapping agent + the direct chat/recommend calls.

```
LLM_PROVIDER=openai                 # gemini (default) | openai | anthropic | azure | …
OPENAI_API_KEY=sk-...               # or ANTHROPIC_API_KEY=...
LLM_MODEL_DEFAULT=gpt-4o            # optional global model
LLM_MODEL_CUSTOMER_SUPPLY=gpt-4o    # optional per-agent: LLM_MODEL_<AGENT>
```

### Prerequisite — `litellm` must be in the image

Non-Gemini providers use `litellm`, installed **non-fatally** in `backend/Dockerfile` (a Gemini build can never break on it). To use OpenAI it must be present:

- Check the build log — a `WARN: litellm not installed under current pins` line means it did not install.
- To require it: add `litellm>=1.40,<2.0` to `backend/code/orchestrator_service/requirements.txt` and rebuild.

### Where to set it

- **Local:** add the vars to the backend `environment:` block in `docker-compose.yml`, then `docker compose up --build`.
- **GCP:** export them before the deploy script (pass-through is wired): `LLM_PROVIDER=openai OPENAI_API_KEY=sk-... bash deploy-gcp.sh`

> **Verify:** `curl <url>/api/health` → `"provider": "openai"`. **Production keys:** use Secret Manager (`--set-secrets=OPENAI_API_KEY=…`), not plaintext env. FinOps cost rows exist for `gpt-4o(/mini)` and `claude-3-5-sonnet/haiku`.

## 7 Platform support & extending to AWS/Azure

| PLATFORM | COMPUTE | SILVER DATA | STATUS |
|---|---|---|---|
| **GCP** | Cloud Run | BigQuery | **Done · verified** |
| **Databricks** (AWS/Azure/GCP-hosted) | Databricks Apps | Unity Catalog | **Mapping+views done · read-path Milestone B** |
| **AWS native** | ECS / App Runner | Redshift / Snowflake | **Extension** |
| **Azure native** | Container Apps | Synapse / Fabric | **Extension** |

**Adding a platform = 3 pieces, no app rewrite:** (1) `deployment/config/<platform>/silver.config.json`; (2) `backend/code/platform_adapters/<platform>_adapter.py` implementing the 5-method `CatalogAdapter` + register in `get_adapter()`; (3) a deploy branch in `driver.py`.

## 8 Verification status

| CHECK | METHOD | RESULT |
|---|---|---|
| Backend modules + scripts compile | py_compile | PASS |
| LLM switch — gemini default → openai/anthropic, per-agent overrides | host test | PASS |
| Data seam — default + `SEMANTIC_DS` override | live BigQuery | PASS · 34/569 |
| Mapping introspect via runner | live BigQuery | PASS · 34/569 |
| Driver dispatch (gcp / local) | dry-run | PASS |
| Backend image build (litellm install) | docker build | RUN AT DEPLOY (non-fatal) |
| Databricks app read-path | — | MILESTONE B |

## 9 Implementation reference (file map)

### New modules (app-imported, in `backend/code/`)

`orchestrator_service/silver_target.py` · `orchestrator_service/model_provider.py` · `platform_adapters/{__init__,base,bigquery_adapter,databricks_adapter}.py` · `mapping_tools.py`

### Minimal core edits (additive, backwards-compatible)

| FILE | CHANGE |
|---|---|
| `agent_tools.py` | import `SEMANTIC_DS` / `DECISIONS_DS` from the seam (defensive fallback) |
| `data_pipeline.py` | seam import; `fetch_data_dictionary` uses `{semantic_dataset()}` ; `MODEL_PRICES` + cross-provider `_est_cost` |
| `agents.py` | 6 agents build via `build_model()` ; `schema_mapping` agent registered |
| `main.py` | direct chat / fulfillment calls via `model_provider.complete()` ; `/health` reads `LLM_PROVIDER` |
| `backend/Dockerfile` | COPY the new modules; non-fatal constrained `litellm` install |
| `deploy.sh` · `deploy-gcp.sh` · `cloudbuild.yaml` | thread `LLM_*` env; cloudbuild `AI_PROVIDER=claude` landmine fixed → Gemini default |

## 10 Status / out of scope

- **GCP** end-to-end & verified. **Databricks** mapping + view creation done; the app's read-path against Unity Catalog is **Milestone B** (the seam is in place).
- Gemini is the default everywhere; non-Gemini providers need `litellm` installed + a key.
- **Out of scope:** live SAP execution (still mocked); stale `tiger_semantic` snapshots (data-team refresh).

---